DETAILED PROJECT REPORT (INT428)

Project Title: AI-Powered LPU Campus Treasure Hunt Platform
Domain: Domain-Specific Generative AI Chatbot Using APIs

---

1. Abstract

The AI-Powered LPU Campus Treasure Hunt Platform is a domain-specific web application designed to gamify campus exploration. Built using a modern technology stack including Python (Flask), HTML5/CSS3, JavaScript, and Leaflet.js, the project integrates Google's Generative AI (Gemini 1.5 Flash) to dynamically generate contextual hints and provide domain-constrained guidance. By strictly configuring model parameters such as Temperature and Top-p, the integrated chatbot acts exclusively as an LPU domain expert. The system utilizes a stateless HTTP architecture with persistent JSON session tracking, allowing users to navigate through a series of locational riddles across the university campus. This report details the system architecture, implementation logic, and the practical integration of Generative AI to solve the real-world problem of engaging, autonomous campus navigation.

---

2. Introduction

2.1 Background

Navigating a large and complex university campus like Lovely Professional University (LPU) can be overwhelming for new students and visitors. Traditional campus maps are static and offer little engagement. To solve this, gamification techniques can be applied to transform navigation into an interactive experience.

2.2 Problem Statement

There is a lack of interactive, intelligent systems that guide users around the campus while keeping them engaged. A static list of locations does not encourage exploration. Furthermore, building a dynamic hint system traditionally requires massive, hardcoded databases for every possible user query.

2.3 Proposed Solution

We propose a web-based "Treasure Hunt" platform where users must solve riddles to find specific campus locations (e.g., Admissions Block, Unipolis, Central Library). To handle dynamic user interactions and provide hints, the system integrates a Generative AI chatbot customized specifically for the LPU domain.

---

3. Project Objectives

1. **Interactive Navigation:** Develop a dynamic, map-based web application using Leaflet.js.
2. **Generative AI Integration:** Utilize the Google Gemini API to serve as an intelligent, responsive "Tour Guide."
3. **Domain Constraint:** Apply strict Prompt Engineering to ensure the AI strictly acts as an LPU Domain Expert and refuses out-of-domain queries.
4. **Model Configuration:** Demonstrate control over AI output behavior by manually tuning `Temperature` and `Top-p` parameters based on the specific context of the request.
5. **Premium User Interface:** Implement a responsive, dark-themed UI featuring advanced CSS-coded architectural art (mimicking the LPU Admissions Block) instead of static images.

---

4. System Architecture & Technology Stack

The project follows a Client-Server architecture deployed via Serverless functions.

4.1 Technology Stack

- **Frontend:** HTML5, CSS3 (Vanilla), JavaScript (ES6+).
- **Mapping Engine:** Leaflet.js with OpenStreetMap tiles (custom dark-mode CSS filters applied).
- **Backend:** Python 3.9, Flask Framework.
- **Database:** Persistent JSON file storage (`database.json`) for lightweight session management.
- **AI Provider:** Google Generative AI API (`gemini-1.5-flash`).
- **Deployment:** Vercel (using `@vercel/python` builder via `vercel.json`).

4.2 Data Flow Architecture

1. **Client Request:** The user interacts with the UI (e.g., submitting a guess for a location pin or asking the chatbot for a hint).
2. **Route Handling:** The Flask backend receives the POST request containing the `user_id` and payload.
3. **State Validation:** The backend reads `database.json` to verify the user's current level and game state.
4. **AI Invocation:** If the request requires AI processing (a failed guess or a chat message), Flask constructs a domain-specific prompt and invokes the Gemini API.
5. **Response Delivery:** The API returns the generated text, which Flask wraps in a JSON response and sends back to the client.
6. **UI Update:** JavaScript updates the DOM, pans the map, or displays the chatbot response.

---

5. Implementation Details

5.1 Generative AI Integration & Configuration

To meet the assessment criteria for a "Domain-Specific Chatbot," the AI integration required strict configuration.

A. Strict Domain Prompting (Role Assignment) The system prompt was engineered to force the LLM into a specific persona:

> *"System Instruction: You are a helpful and mysterious tour guide for the LPU (Lovely Professional University) Campus Treasure Hunt. Your role is 'Domain Expert'. You must ONLY answer questions related to LPU, the campus, locations, or the treasure hunt game. If the user asks something outside of this domain, politely refuse..."*

B. Model Parameter Tuning The system dynamically adjusts model parameters based on the required task:

- **Factual Chat (Temperature = 0.4, Top-p = 0.8):** Used for general user queries. A lower temperature ensures the model remains deterministic, factual, and strictly adheres to the domain constraints without hallucinating.
- **Cryptic Hints (Temperature = 0.8, Top-p = 0.9):** Used when automatically generating location hints. A higher temperature allows the model to be more creative, ensuring hints are appropriately mysterious and varied.

5.2 Frontend Engineering

- **CSS Architectural Art:** Instead of using an `<img>` tag for the homepage, the LPU Admissions block was recreated using pure HTML `<div>` elements and advanced CSS (`clip-path` , `transform: skew` , and z-index layering). This creates a lightweight, highly responsive, and premium dark-mode landing page.
- **Interactive Map:** Leaflet.js is utilized to plot coordinates for LPU landmarks (e.g., Block 32, Uni Mall). Markers are dynamically styled based on user progression (Locked, Active, Solved).

5.3 Backend Logic & Session Management

The Flask backend uses a simple, effective JSON-based key-value store to track users. Each user is assigned a unique `uid` stored in `localStorage` . The backend tracks their `level` , `start_time` , `end_time` , and `hints_used` .

This data is used to calculate the final leaderboard score: `Final Score = Total Time Taken + (Hints Used * 300 seconds penalty)`

---

6. Features & Functionalities

1. **Real-Time Location Validation:** Users must guess the name of the location based on the active pin's riddle. String normalization (removing spaces/punctuation) is used to validate answers gracefully.
2. **Automated AI Hint System:** Answering incorrectly automatically slides open the chatbot sidebar and triggers a dynamically generated AI hint.
3. **Dynamic Leaderboard:** A dedicated `/leaderboard` route calculates scores and ranks teams based on speed and reliance on AI hints.
4. **Seamless UI Transitions:** The CSS-coded landing page features a "Click to Start" mechanic that fades out smoothly to reveal the interactive game map.

---

7. Conclusion & Future Scope

7.1 Conclusion

The project successfully demonstrates the practical application of Generative AI in a localized, domain-specific context. By integrating the Gemini API with strict prompt engineering and parameter configurations, the platform provides an intelligent, engaging, and highly constrained user experience. The combination of CSS art, interactive mapping, and LLM capabilities results in a premium, production-ready prototype.

7.2 Future Scope

- **GPS Geofencing:** Replace manual text guessing with HTML5 Geolocation, requiring users to physically stand at the coordinates to unlock the pin.
- **Multimedia Rewards:** Display actual photographs of the LPU locations inside a success modal upon solving a riddle.
- **Multilingual Support:** Leverage the LLM's natural translation capabilities to allow users to play the treasure hunt in Hindi or Punjabi.

End of Report